The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch11_interpretability

AYuSh

Contents

Chapter 11: Model Interpretability & Explainability

A model that predicts well but cannot be explained is a liability: regulators demand reasons for credit denials, doctors will not act on an oracle, debugging a silent failure requires knowing what the model attends to, and stakeholders who cannot interrogate a model will not trust it — often correctly, since Chapter 9's leakage stories showed that a suspiciously strong model is usually strong for the wrong reason. Interpretability is the toolkit for opening the box: measuring which features drive predictions overall (global), why *this* prediction came out the way it did (local), what shape each feature's effect takes, and whether the model's behavior differs across protected groups. Interviewers love this chapter's territory because it separates candidates who can call `shap.TreeExplainer` from those who know what the numbers mean, when they lie, and which method answers which question.

The chapter follows the outline's arc. First the map: intrinsic versus post-hoc, model-specific versus model-agnostic, global versus local. Then feature importance and its traps — impurity importance's cardinality bias versus permutation importance (Listing 1), and coefficients that mislead when features are unscaled or collinear (Listing 2). Partial dependence and ICE plots come next, with a staged interaction that a PDP averages into a flat line while ICE curves expose it (Listing 3). The heart of the chapter is SHAP: Shapley values computed exactly from the game-theoretic definition and verified against KernelSHAP (Listing 4), then TreeSHAP on a churn-style model with the additivity check and global aggregation (Listing 5). LIME is built from scratch — perturb, weight, fit a local surrogate (Listing 6). A dedicated section stages how correlated features corrupt every importance method (Listing 7). The chapter closes with fairness: demographic parity, equalized odds, disparate impact, and a demonstration that dropping the protected attribute does not remove bias when a proxy carries it (Listing 8).

The interpretability map: global vs local, intrinsic vs post-hoc

Three axes organize every method in this chapter, and stating them is the expected first move in any interpretability interview answer.

Intrinsic versus post-hoc. An *intrinsically interpretable* model is readable by construction: a linear model's coefficients, a shallow decision tree's paths, a rule list. A *post-hoc* method explains an already-trained black box from the outside — permutation importance, PDP, SHAP, LIME all work this way. The classic trade-off is that intrinsic models buy transparency with capacity, though the gap is smaller than folklore suggests:

on tabular data a well-regularized GBM explained with TreeSHAP is often both more accurate and more *usefully* explained than a long linear model with hundreds of interacting coefficients. Rule of thumb: when the stakes are high and the signal is simple, prefer the intrinsic model; when you need the capacity, pair the black box with post-hoc tools and validate the explanations.

Model-specific versus model-agnostic. Coefficients belong to linear models; impurity importance and TreeSHAP belong to trees; saliency maps belong to differentiable networks. Model-agnostic methods — permutation importance, PDP/ICE, KernelSHAP, LIME — treat the model as a function $f(x)$ you can query, which makes them universal but often slower and dependent on how you generate the query points (the source of most of their failure modes, as Listing 7 shows).

Global versus local. A *global* explanation summarizes the model's overall behavior: which features matter on average, what the aggregate effect of a feature looks like. A *local* explanation accounts for one prediction: why did *this* applicant get denied? The two can disagree instructively — a feature that matters enormously for a handful of rows can look minor globally, and Listing 3's interaction shows a feature with a strong effect on every single row that averages to *zero* globally. SHAP bridges the axes: it is defined locally, and its local values aggregate into faithful global summaries (Listing 5).

The vocabulary that recurs downstream: **faithfulness** (does the explanation reflect what the model actually computes?), **stability** (do reruns give the same explanation?), and **plausibility** (does it match human intuition?) — and the trap that an explanation can be plausible without being faithful, which is worse than no explanation because it manufactures unearned trust.

Feature importance methods

Impurity importance (MDI). Tree ensembles ship with a free importance score: each split reduces impurity (Gini, entropy, or variance), and summing the reduction credited to each feature over all splits, weighted by the fraction of samples reaching the node, gives *mean decrease in impurity*. It costs nothing — the numbers were computed during training — and it is the default `feature_importances_` in scikit-learn. It has two well-known biases. First, the **cardinality bias**: features with many possible split points (continuous features, high-cardinality categoricals, ID-like columns) get more chances to produce a spuriously good split, so pure noise with many unique values earns nonzero importance. Listing 1 stages this: a continuous noise column scores MDI 0.074 — level with two *real* informative features and seven times the score of an equally useless binary noise column. Second, MDI is computed on the **training** data structure, so it reflects what the model used to fit, including what it used to overfit — it says nothing about generalization.

Permutation importance fixes both. Shuffle one column of *held-out* data, breaking its relationship with the target while preserving its marginal distribution, and measure how much the test metric drops: importance = performance lost when the feature's

information is destroyed. It is model-agnostic, evaluated on test data, and immune to cardinality bias — in Listing 1 the same continuous noise column falls to +0.002, statistically zero, while the real features keep their ranking. Repeat the shuffle (`n_repeats`) and report the spread. Its costs: one model evaluation pass per feature per repeat, and a serious failure mode under correlated features — permuting one member of a correlated pair creates rows that cannot occur in reality *and* lets the model recover the signal from the surviving twin, deflating both importances (Listing 7 measures the collapse). **Drop-column importance** — retrain without the feature, measure the drop — is the gold standard for "what does this feature contribute *given the others*," but costs a full retrain per feature and answers a subtly different question: a feature can be individually informative yet have zero drop-column importance because a correlated feature covers for it.

Coefficients as importance. For linear models, $|w_j|$ measures importance only after features share a scale: coefficients have units (target change per *unit* of feature), so a feature measured in dollars gets a coefficient 1,500 times smaller than an equally important feature measured in decades. Listing 2 builds two equally predictive features — income in dollars, age in years — and the raw fit reports coefficients 0.000116 versus 0.176: income looks irrelevant. Standardize both and the truth appears: 1.73 versus 1.75, statistically identical. The second trap is collinearity, inherited from Chapter 5: with two near-duplicate features ($\rho \approx 0.999$), bootstrap refits swing the individual coefficients from (+1.34, +0.35) to (+2.09, −0.39) — one even flips sign — while their *sum* stays a stable ~1.7. Only the combined effect is identified; reading individual coefficients as importances under collinearity is reading noise.

Partial dependence and ICE plots

Importance says *how much* a feature matters; **partial dependence** says *what shape* its effect takes. The partial dependence of f on feature j at value v is the average prediction when feature j is forced to v for everyone:

$$PD_j(v) = \frac{1}{n}\sum_{i=1}^{n} f(x_i^{(-j)},\ x_j = v)$$

where $x_i^{(-j)}$ keeps row i's other features. Sweep v over a grid and plot: the resulting curve reveals linearity, saturation, thresholds, non-monotonicity. Two caveats define the method. First, forcing $x_j = v$ for *everyone* assumes feature j is independent of the rest — with correlated features the average is taken over synthetic rows that cannot exist (a 1.9 m person forced to weigh 45 kg), the same pathology as permutation importance. Second, and the point of Listing 3: **PDP is an average, and averages hide heterogeneity.**

ICE (individual conditional expectation) plots fix the second caveat by not averaging: one curve per row, showing that row's prediction as feature j sweeps the grid — the PDP is exactly the mean of the ICE curves. Listing 3 constructs the canonical failure: $y = 2x_1$

for group B but $y = -2x_1$ for group A. The PDP of x_1 is flat — fitted slope −0.01, "no effect" — while the ICE curves split into two fans with slopes −1.68 and +1.69. A feature with a strong effect on *every single row* averages to nothing because the effects cancel. The practitioner's habit: always plot ICE under (or instead of) the PDP; if the ICE curves are roughly parallel the PDP is a faithful summary, and if they cross or fan out there is an interaction that the PDP is erasing — go find the interacting feature (centered "c-ICE" curves, which pin all curves to zero at the grid's left edge, make the divergence even easier to see).

Shapley values and SHAP

The most principled answer to "how much did each feature contribute to this prediction?" comes from cooperative game theory. Treat the features as players in a coalition game where the payout of a coalition S is the model's expected prediction when only the features in S are known:

$$v(S) = E[f(x_S, X_{\bar{S}})]$$

— features in S fixed to this instance's values, the rest averaged over a background distribution. The **Shapley value** of feature j is its marginal contribution averaged over every possible order in which features could be revealed:

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!\ (|F| - |S| - 1)!}{|F|!} [v(S \cup \{j\}) - v(S)]$$

It is the unique attribution satisfying four axioms: **efficiency** (contributions sum exactly to $f(x) - E[f(X)]$ — the explanation accounts for the whole prediction), **symmetry** (identical contributors get identical credit), **dummy** (a feature that never changes any coalition's value gets zero), and **additivity** (consistent across ensembled models). Efficiency is the property the other methods lack and the one to name in an interview: SHAP values are a complete decomposition of the prediction's distance from the base rate, not just a ranking. Listing 4 implements the definition literally — all 2^d coalitions, factorial weights — and the additivity check lands exactly: base + contributions = 5.213 = $f(x)$ to the third decimal.

The definition costs $O(2^d)$, hence the approximations. **KernelSHAP** (model-agnostic) exploits the fact that Shapley values are the solution to a specific weighted linear regression on coalition indicators: sample coalitions, evaluate $v(S)$ by marginalizing over a background dataset, and solve the regression with the Shapley kernel weights. Listing 4 confirms the equivalence — with enough samples KernelSHAP matches the exact enumeration to four decimal places (max gap 0.0000). It works on any model but each explanation costs (samples × background) model evaluations; explaining a large test set can be slower than training. **TreeSHAP** (model-specific) computes *exact* Shapley values for tree ensembles in polynomial time — $O(TLD^2)$ per instance, with T trees, L leaves, depth D — by pushing coalitions through the tree structure analytically. That is the

practical reason SHAP dominates tabular ML: for GBMs and forests you get axiomatic attributions at production speed. Listing 5 runs it on a churn-style GBM: the SHAP matrix for 1,500 test rows arrives in seconds, one customer's prediction decomposes into base −0.012 plus contributions −4.911 = −4.924 log-odds → p = 0.007, matching `predict_proba` exactly, with `contract_len` (−3.614) doing most of the pushing. Note the unit: for classifiers TreeSHAP explains the *margin* (log-odds), not the probability — contributions are additive in log-odds space and only there.

From local to global. Because every row gets an exact decomposition, global importance is just aggregation: mean |SHAP| per feature. Listing 5 shows the mean-|SHAP| ranking and the permutation ranking agreeing feature-for-feature — reassuring when it happens, diagnostic when it does not (disagreement usually points at correlated features or at a feature that matters hugely for a few rows). The `shap` library's summary ("beeswarm") plot layers more: each dot a row, x-position the SHAP value, color the feature's value — so you read magnitude, direction, and nonlinearity in one figure. SHAP's caveats: computed against a **background/reference** distribution whose choice changes the values (explaining "versus the average customer" differs from "versus approved customers"); interventional marginalization inherits the impossible-points problem under correlated features (Listing 7); and SHAP explains *the model*, not *the world* — a feature can get large SHAP values because the model leans on it as a proxy, not because it causes anything.

LIME: local surrogate models

LIME (Local Interpretable Model-agnostic Explanations) answers the local question with a different bargain: approximate the black box *near this instance* with a model simple enough to read. Formally it minimizes $\mathcal{L}(f, g, \pi_x) + \Omega(g)$ over a family G of interpretable models — find the surrogate g (usually sparse linear) that best matches f on points weighted by proximity π_x to the instance, penalizing complexity. The algorithm, which Listing 6 implements in ~15 lines: (1) sample perturbations around x (Gaussian noise in standardized space for tabular data; masking words or superpixels for text and images); (2) query the black box for each perturbation; (3) weight each perturbed point by an exponential proximity kernel; (4) fit a weighted ridge regression — its coefficients are the explanation.

Listing 6 explains a decision-boundary customer (p = 0.499) and gets a readable answer: per standard deviation, `contract_len` pushes −0.278, `monthly_fee` +0.190, with a weighted local fit R^2 of 0.623. Two practical lessons are baked in. First, the *choice of instance matters*: explaining a saturated prediction (p ≈ 0.003) yields a nearly flat local surface and a meaningless surrogate — LIME explanations are only as informative as the local gradient. Second, *stability must be checked, never assumed*: LIME's perturbations are random, so reruns can reorder features; with 5,000 perturbations the top feature is stable across five seeds here, but the honest workflow always reruns with different seeds and reports whether the story holds. LIME's knobs — kernel width, number of

perturbations, surrogate sparsity — are unprincipled in the sense that no axiom picks them, and the perturbation distribution can wander off the data manifold. Versus SHAP: LIME is faster per explanation than KernelSHAP and more flexible across modalities (its image/text variants are natural), but its attributions satisfy no efficiency axiom — they are local slopes, not a decomposition of the prediction — and are generally less stable. When both are cheap (trees → TreeSHAP), SHAP wins; LIME earns its keep on expensive black boxes and non-tabular inputs.

When explanations mislead: correlated features

Every post-hoc method in this chapter queries the model on synthetic points, and correlated features make those points lies. Listing 7 duplicates a feature ($\rho = 1.000$) and watches the damage. Permutation importance of x_1 collapses from 1.638 to 0.478 — not because x_1 matters less but because when it is shuffled, its twin still carries the signal, so performance barely drops; read naively, "importance 0.478" understates the causal role by 3.4×. SHAP splits the credit instead: mean |SHAP| 1.659 alone becomes 0.864 + 0.797 for the pair — the *sum* is conserved (the efficiency axiom guarantees the total decomposition), but each individual number halves. Neither behavior is a bug; they are different, defensible answers to "who gets credit for shared information" — and both surprise anyone expecting the duplicate to be free. The listing's last lines expose the deeper mechanism: after permuting one twin, the correlation between the two drops from +1.000 to −0.024, i.e. the method is scoring the model on rows where near-identical features disagree wildly — a region the model never saw and reality never produces.

The defenses: cluster correlated features and permute/interpret them as groups; use drop-column importance when you can afford retrains; prefer conditional/observational SHAP variants when the independence assumption is untenable (knowing they trade one subtlety for another — credit then leaks to correlated bystanders); and above all report correlation structure alongside any importance table. The interview sound bite: *feature importance is well-defined only up to the correlation structure of the features* — any single number per feature is a projection of a joint story.

Fairness and bias detection

Interpretability's highest-stakes application is auditing models for discrimination. The vocabulary: a **protected attribute** A (race, gender, age…), a prediction $\hat{Y}$, an outcome Y, and *group fairness criteria* that compare error or selection rates across groups. **Demographic parity** demands equal selection rates: $P(\hat{Y} = 1 \mid A = 0) = P(\hat{Y} = 1 \mid A = 1)$; its ratio form is **disparate impact**, with the legal "80% rule" flagging ratios below 0.8. **Equalized odds** demands equal TPR *and* FPR across groups — errors, not just selections, must be balanced; **equal opportunity** relaxes it to TPR only (qualified candidates get equal chances). **Calibration within groups** demands that a score of 0.7 mean 70% in every group. A foundational impossibility theorem (Kleinberg et al.;

Chouldechova): when base rates differ across groups, calibration and equalized odds cannot hold simultaneously except in degenerate cases — *fairness metrics are choices with trade-offs, not boxes to tick all at once*, and stating this is the strongest single sentence available in a fairness interview.

The most common misconception is **fairness through unawareness** — "we don't use race as a feature, so the model can't discriminate." Listing 8 dismantles it: a hiring model trained *without* the protected attribute, on historical labels where 45% of group-0's qualified candidates were erased (biased past decisions), with a zip-code proxy that strongly encodes group. The model reconstructs the bias through the proxy: selection rates 0.273 vs 0.331, disparate impact 0.82 (hovering at the legal line), and against *true* merit a TPR gap of +0.093 — qualified group-0 candidates are 9 points less likely to be selected. The listing also teaches the subtler lesson: audited against the *historical labels*, the FPR gap (−0.108) looks like the model over-selects group 0 — the audit's ground truth is itself contaminated, and **a fairness audit is only as good as the labels it audits against**. Mitigations come in three families: *pre-processing* (reweigh or repair the training data, remove proxies — hard, since proxies hide), *in-processing* (fairness constraints or adversarial debiasing in the loss), *post-processing* (group-specific thresholds). Listing 8 applies the cheapest, post-processing to equalize opportunity: per-group thresholds chosen to hit TPR ≈ 0.80 on merit in both groups collapse the TPR gap to −0.006 and, here, drag demographic parity to 0.98 as a side effect. SHAP plugs in upstream of all of this: computing SHAP values for the proxy feature by group reveals *how* the model uses the proxy, turning "the metric is bad" into "this feature carries the bias."

Code implementations

Listing 1 — Impurity vs permutation importance: the cardinality trap

```
"""Listing 1: impurity importance vs permutation importance -- the cardinality trap."""
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification

rng = np.random.default_rng(0)
X, y = make_classification(n_samples=4000, n_features=5, n_informative=3,
                           n_redundant=0, random_state=0)
df = pd.DataFrame(X, columns=[f"real_{i}" for i in range(5)])
# a pure-noise feature with many unique values (like an ID or a timestamp)
df["noise_hicard"] = rng.normal(size=len(df))          # continuous noise
# a pure-noise feature with only 2 levels
df["noise_binary"] = rng.integers(0, 2, size=len(df))  # binary noise

Xtr, Xte, ytr, yte = train_test_split(df, y, random_state=0)
rf = RandomForestClassifier(n_estimators=300, random_state=0).fit(Xtr, ytr)
print(f"test accuracy: {rf.score(Xte, yte):.3f}")

# 1) impurity-based (Gini) importance -- computed on TRAINING data structure
```

```
imp = pd.Series(rf.feature_importances_, index=df.columns).sort_values(ascending=False)
print("\nimpurity (MDI) importance:")
for k, v in imp.items():
    print(f"  {k:<14}{v:.4f}")

# 2) permutation importance -- computed on HELD-OUT data
pi = permutation_importance(rf, Xte, yte, n_repeats=20, random_state=0)
ps = pd.Series(pi.importances_mean, index=df.columns).sort_values(ascending=False)
print("\npermutation importance (test):")
for k, v in ps.items():
    print(f"  {k:<14}{v:+.4f}")
```

Output:

```
test accuracy: 0.849

impurity (MDI) importance:
  real_1        0.3766
  real_4        0.2100
  real_3        0.1788
  real_2        0.0763
  noise_hicard  0.0740
  real_0        0.0739
  noise_binary  0.0105

permutation importance (test):
  real_1        +0.2762
  real_4        +0.1315
  real_3        +0.1163
  real_0        +0.0054
  real_2        +0.0052
  noise_binary  +0.0024
  noise_hicard  +0.0021
```

The continuous noise column earns MDI 0.0740 — indistinguishable from two genuinely informative features and 7× the equally-useless binary noise, purely because it offers thousands of candidate split points. Permutation importance on held-out data sends both noise columns to ~0.002 and preserves the real features' ranking.

Listing 2 — Coefficients as importance: scale and collinearity traps

```
"""Listing 2: coefficients as importance -- scale and collinearity traps."""
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(1)
n = 5000
income_usd = rng.normal(60_000, 15_000, n)     # dollars: huge scale
age_years  = rng.normal(40, 10, n)             # years: small scale
# both matter EQUALLY in standardized terms
z = 1.0 * (income_usd - 60_000) / 15_000 + 1.0 * (age_years - 40) / 10
y = (z + rng.normal(0, 1, n) > 0).astype(int)

X = np.c_[income_usd, age_years]
raw = LogisticRegression(max_iter=5000).fit(X, y)
print("raw coefficients   :", np.round(raw.coef_[0], 6),
      "-> income looks ~0, age looks big")

Xs = StandardScaler().fit_transform(X)
std = LogisticRegression(max_iter=5000).fit(Xs, y)
```

```
print("scaled coefficients:", np.round(std.coef_[0], 3),
      "-> equal importance revealed")

# collinearity: two near-duplicate features split (or flip) the credit
x1 = rng.normal(size=n)
x2 = x1 + rng.normal(0, 0.05, n)              # rho ~ 0.999
y2 = (x1 + rng.normal(0, 1, n) > 0).astype(int)
for i in range(3):                            # three bootstrap refits
    idx = rng.integers(0, n, n)
    m = LogisticRegression(max_iter=5000).fit(np.c_[x1, x2][idx], y2[idx])
    print(f"bootstrap {i}: coef(x1)={m.coef_[0][0]:+.2f}  coef(x2)={m.coef_[0][1]:+.
2f}"
          f"  sum={m.coef_[0].sum():+.2f}")
```

Output:

```
raw coefficients   : [1.16000e-04 1.76346e-01] -> income looks ~0, age looks big
scaled coefficients: [1.732 1.749] -> equal importance revealed
bootstrap 0: coef(x1)=+1.34  coef(x2)=+0.35  sum=+1.69
bootstrap 1: coef(x1)=+2.09  coef(x2)=-0.39  sum=+1.70
bootstrap 2: coef(x1)=+1.88  coef(x2)=-0.03  sum=+1.85
```

Two equally important features differ by a factor of 1,500 in raw coefficient purely because of units; standardization reveals the tie (1.732 vs 1.749). Under near-perfect collinearity, individual coefficients swing wildly across bootstrap refits — one flips sign — while their sum stays a stable ~1.7: only the combined effect is identified.

Listing 3 — PDP and ICE from scratch: averages hide interactions

```
"""Listing 3: partial dependence + ICE from scratch -- averages hide interactions."""
import numpy as np
from sklearn.ensemble import GradientBoostingRegressor

rng = np.random.default_rng(2)
n = 3000
x1 = rng.uniform(-2, 2, n)
x2 = rng.integers(0, 2, n)                # binary group flag
x3 = rng.normal(size=n)
# interaction: x1 helps group 1, hurts group 0 -- effects cancel on average
y = np.where(x2 == 1, 2.0 * x1, -2.0 * x1) + 0.5 * x3 + rng.normal(0, 0.3, n)
X = np.c_[x1, x2, x3]
gbm = GradientBoostingRegressor(random_state=0).fit(X, y)
print(f"train R^2: {gbm.score(X, y):.3f}")

def pdp_ice(model, X, feat, grid):
    """Return (pdp, ice): ice[i,g] = f(x_i with feature `feat` set to grid[g])."""
    ice = np.empty((len(X), len(grid)))
    for g, v in enumerate(grid):
        Xmod = X.copy()
        Xmod[:, feat] = v                 # intervene: set feature for EVERYONE
        ice[:, g] = model.predict(Xmod)
    return ice.mean(axis=0), ice          # PDP = average of ICE curves

grid = np.linspace(-2, 2, 9)
pdp, ice = pdp_ice(gbm, X, feat=0, grid=grid)
print("\ngrid            :", np.round(grid, 1))
print("PDP (average)   :", np.round(pdp, 2), "<- looks FLAT: 'x1 has no effect'")

g0, g1 = ice[x2 == 0].mean(axis=0), ice[x2 == 1].mean(axis=0)
print("ICE mean, x2=0  :", np.round(g0, 2), "<- steep negative")
print("ICE mean, x2=1  :", np.round(g1, 2), "<- steep positive")
```

```
slope = np.polyfit(grid, pdp, 1)[0]
s0, s1 = np.polyfit(grid, g0, 1)[0], np.polyfit(grid, g1, 1)[0]
print(f"\nslopes: PDP {slope:+.2f} vs group0 {s0:+.2f} vs group1 {s1:+.2f}")
```

Output:

```
train R^2: 0.939

grid            : [-2.  -1.5 -1.  -0.5  0.   0.5  1.   1.5  2. ]
PDP (average)   : [-0.03  0.07  0.05  0.05  0.07  0.07 -0.01 -0.04 -0.01] <- looks
FLAT: 'x1 has no effect'
ICE mean, x2=0  : [ 3.57  2.31  1.35  0.98  0.14 -0.83 -1.67 -2.04 -3.77] <- steep
negative
ICE mean, x2=1  : [-3.72 -2.23 -1.29 -0.91 -0.    1.    1.68  2.    3.83] <- steep
positive

slopes: PDP -0.01 vs group0 -1.68 vs group1 +1.69
```

The feature x_1 has a strong effect on every row — slope -1.68 in one group, $+1.69$ in the other — yet the PDP's fitted slope is -0.01. An interaction the PDP averages into invisibility is fully exposed the moment the ICE curves are split.

Listing 4 — Exact Shapley values from the definition, verified against KernelSHAP

```
"""Listing 4: exact Shapley values from the definition, verified against KernelSHAP."""
import itertools, math
import numpy as np
from sklearn.ensemble import RandomForestRegressor
import shap

rng = np.random.default_rng(3)
n, d = 2000, 4
X = rng.normal(size=(n, d))
y = 3*X[:, 0] + 2*X[:, 1]*X[:, 2] + rng.normal(0, 0.1, n)   # interaction inside
model = RandomForestRegressor(n_estimators=200, random_state=0).fit(X, y)

background = X[rng.choice(n, 100, replace=False)]            # reference sample
x = X[0]                                                     # instance to explain

def value(S):
    """v(S) = E[f(x_S, X_rest)]: features in S fixed to x, rest drawn from
background."""
    Xb = background.copy()
    Xb[:, list(S)] = x[list(S)]
    return model.predict(Xb).mean()

def exact_shapley(d):
    phi = np.zeros(d)
    for j in range(d):
        others = [k for k in range(d) if k != j]
        for r in range(d):                                   # all coalition sizes
            for S in itertools.combinations(others, r):
                w = math.factorial(len(S)) * math.factorial(d - len(S) - 1) /
math.factorial(d)
                phi[j] += w * (value(S + (j,)) - value(S))   # marginal contribution
    return phi

phi = exact_shapley(d)
base = value(())                                             # E[f(X)]
print("exact Shapley phi :", np.round(phi, 3))
```

```
print(f"base + sum(phi) = {base + phi.sum():.3f}  vs  f(x) = {model.predict([x])[0]:.
3f}"
      "   <- local accuracy (additivity)")

ks = shap.KernelExplainer(model.predict, background)
phi_ks = ks.shap_values(x, nsamples=2000, silent=True)
print("KernelSHAP phi    :", np.round(phi_ks, 3))
print(f"max |exact - kernel| = {np.abs(phi - phi_ks).max():.4f}")
```

Output:

```
exact Shapley phi : [ 5.15  -0.279 -0.298 -0.132]
base + sum(phi) = 5.213  vs  f(x) = 5.213   <- local accuracy (additivity)
KernelSHAP phi    : [ 5.15  -0.279 -0.298 -0.132]
max |exact - kernel| = 0.0000
```

Thirty lines implement the game-theoretic definition — every coalition, factorial weights, marginalization over a background sample. The efficiency axiom lands exactly (5.213 = 5.213), and KernelSHAP's regression-based approximation reproduces the exact enumeration to four decimals.

Listing 5 — TreeSHAP on a GBM: local explanation, additivity, global aggregation

```
"""Listing 5: TreeSHAP on a GBM -- local explanations, additivity, global
aggregation."""
import numpy as np
import pandas as pd
import shap
from sklearn.datasets import make_classification
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split
from sklearn.inspection import permutation_importance

cols = ["tenure", "monthly_fee", "support_calls", "usage_gb", "age", "contract_len"]
X, y = make_classification(n_samples=6000, n_features=6, n_informative=4,
                           n_redundant=1, random_state=7)
X = pd.DataFrame(X, columns=cols)
Xtr, Xte, ytr, yte = train_test_split(X, y, random_state=0)
gbm = GradientBoostingClassifier(random_state=0).fit(Xtr, ytr)
print(f"test accuracy: {gbm.score(Xte, yte):.3f}")

expl = shap.TreeExplainer(gbm)              # polynomial-time exact for trees
base_val = float(np.ravel(expl.expected_value)[0])
sv = expl.shap_values(Xte)                  # (n_test, d) in log-odds space
print(f"shap matrix shape: {sv.shape}, base value (mean log-odds): {base_val:.3f}")

# local: one customer, additivity check in log-odds
i = 5
margin = base_val + sv[i].sum()
print(f"\ncustomer {i}: base {base_val:+.3f} + contributions {sv[i].sum():+.3f}"
      f" = {margin:+.3f} log-odds -> p = {1/(1+np.exp(-margin)):.3f}"
      f" (model says {gbm.predict_proba(Xte.iloc[[i]])[0,1]:.3f})")
contrib = pd.Series(sv[i], index=cols).sort_values(key=np.abs, ascending=False)
for k, v in contrib.items():
    print(f"  {k:<13}{v:+.3f}")

# global = aggregate of locals: mean |SHAP| vs permutation importance rankings
gshap = pd.Series(np.abs(sv).mean(axis=0), index=cols).sort_values(ascending=False)
pi = permutation_importance(gbm, Xte, yte, n_repeats=10, random_state=0)
```

```
gperm = pd.Series(pi.importances_mean, index=cols).sort_values(ascending=False)
print("\nglobal mean|SHAP| ranking:", list(gshap.index))
print("permutation ranking      :", list(gperm.index))
```

Output:

```
test accuracy: 0.932
shap matrix shape: (1500, 6), base value (mean log-odds): -0.012

customer 5: base -0.012 + contributions -4.911 = -4.924 log-odds -> p = 0.007 (model
says 0.007)
  contract_len -3.614
  support_calls-0.931
  age          -0.237
  monthly_fee  -0.147
  usage_gb     +0.022
  tenure       -0.005

global mean|SHAP| ranking: ['contract_len', 'monthly_fee', 'support_calls', 'age',
'usage_gb', 'tenure']
permutation ranking      : ['contract_len', 'monthly_fee', 'support_calls', 'age',
'usage_gb', 'tenure']
```

One customer's 0.007 churn probability decomposes exactly: base log-odds −0.012 plus feature contributions −4.911, dominated by `contract_len` (−3.614). Aggregating |SHAP| across 1,500 rows gives a global ranking that matches permutation importance feature-for-feature.

Listing 6 — LIME from scratch: perturb, weight, fit a local linear model

```
"""Listing 6: LIME from scratch -- perturb, weight by proximity, fit a local linear
model."""
import numpy as np
import pandas as pd
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.linear_model import Ridge
from sklearn.datasets import make_classification
from sklearn.preprocessing import StandardScaler

cols = ["tenure", "monthly_fee", "support_calls", "usage_gb", "age", "contract_len"]
X, y = make_classification(n_samples=6000, n_features=6, n_informative=4,
                           n_redundant=1, random_state=7)
model = GradientBoostingClassifier(random_state=0).fit(X, y)
scaler = StandardScaler().fit(X)

def lime_explain(x, n_samples=5000, kernel_width=1.5, seed=0):
    rng = np.random.default_rng(seed)
    d = len(x)
    # 1) perturb around x in STANDARDIZED space (Gaussian cloud)
    Z_std = scaler.transform([x]).repeat(n_samples, axis=0) \
            + rng.normal(0, 1, (n_samples, d))
    Z = scaler.inverse_transform(Z_std)
    # 2) query the black box on the perturbations
    f = model.predict_proba(Z)[:, 1]
    # 3) proximity kernel: nearby points dominate the local fit
    dist = np.linalg.norm(Z_std - scaler.transform([x]), axis=1)
    w = np.exp(-dist**2 / kernel_width**2)
    # 4) weighted linear surrogate = the explanation
    lin = Ridge(alpha=1.0).fit(Z_std, f, sample_weight=w)
    return lin.coef_, lin.intercept_, lin.score(Z_std, f, sample_weight=w)
```

```
p_all = model.predict_proba(X)[:, 1]
x = X[np.argmin(np.abs(p_all - 0.5))]
# explain a BOUNDARY case, not a saturated one
coef, b, r2 = lime_explain(x)
print(f"model p(churn|x) = {model.predict_proba([x])[0,1]:.3f}")
print(f"local surrogate fit R^2 (weighted): {r2:.3f}\n")
expl = pd.Series(coef, index=cols).sort_values(key=np.abs, ascending=False)
print("LIME local coefficients (per +1 std of feature):")
for k, v in expl.items():
    print(f"  {k:<14}{v:+.4f}")

# instability check: rerun with different perturbation seeds
tops = [pd.Series(lime_explain(x, seed=s)[0], index=cols)
          .abs().idxmax() for s in range(5)]
print("\ntop feature across 5 seeds:", tops)
```

Output:

```
model p(churn|x) = 0.499
local surrogate fit R^2 (weighted): 0.623

LIME local coefficients (per +1 std of feature):
  contract_len  -0.2778
  monthly_fee   +0.1896
  age           -0.1462
  usage_gb      -0.0466
  tenure        +0.0141
  support_calls -0.0058

top feature across 5 seeds: ['contract_len', 'contract_len', 'contract_len',
'contract_len', 'contract_len']
```

Fifteen lines of algorithm: Gaussian perturbations, black-box queries, proximity weights, weighted ridge. At the decision boundary the surrogate reads cleanly (`contract_len` −0.278 per std) and the top feature survives five reruns. Explaining a saturated instance ($p \approx 0.003$) instead yields a near-flat local surface and R^2 of 0.07 — LIME is only as informative as the local gradient.

Listing 7 — Correlated features break importance: credit splitting and impossible points

```
"""Listing 7: correlated features break importance -- credit splitting and impossible
points."""
import numpy as np
import pandas as pd
import shap
from sklearn.ensemble import RandomForestRegressor
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(4)
n = 3000
x1 = rng.normal(size=n)
dup = x1 + rng.normal(0, 0.01, n)          # near-perfect copy of x1
x3 = rng.normal(size=n)
y = 2.0 * x1 + 1.0 * x3 + rng.normal(0, 0.2, n)

for name, feats in [("without duplicate", {"x1": x1, "x3": x3}),
                    ("with duplicate   ", {"x1": x1, "x1_dup": dup, "x3": x3})]:
    X = pd.DataFrame(feats)
```

```
    Xtr, Xte, ytr, yte = train_test_split(X, y, random_state=0)
    rf = RandomForestRegressor(n_estimators=100, random_state=0).fit(Xtr, ytr)
    pi = permutation_importance(rf, Xte, yte, n_repeats=10, random_state=0)
    sv = shap.TreeExplainer(rf).shap_values(Xte)
    ms = np.abs(sv).mean(axis=0)
    row = "  ".join(f"{c}: perm {p:.3f} shap {s:.3f}"
                    for c, p, s in zip(X.columns, pi.importances_mean, ms))
    print(f"{name}  {row}")

# why permuting correlated features is dangerous: it manufactures impossible rows
X2 = pd.DataFrame({"x1": x1, "x1_dup": dup, "x3": x3})
Xp = X2.copy(); Xp["x1"] = rng.permutation(Xp["x1"].values)
print(f"\ncorr(x1, x1_dup) real data     : {X2['x1'].corr(X2['x1_dup']):+.3f}")
print(f"corr(x1, x1_dup) after permute : {Xp['x1'].corr(Xp['x1_dup']):+.3f}"
      "  <- evaluates the model on points that cannot exist")
```

Output:

```
without duplicate  x1: perm 1.638 shap 1.659  x3: perm 0.352 shap 0.796
with duplicate     x1: perm 0.478 shap 0.864  x1_dup: perm 0.377 shap 0.797  x3: perm
0.352 shap 0.793

corr(x1, x1_dup) real data     : +1.000
corr(x1, x1_dup) after permute : -0.024  <- evaluates the model on points that cannot
exist
```

Adding a duplicate collapses x_1's permutation importance 1.638 → 0.478 (the twin covers for it) while SHAP splits the credit 1.659 → 0.864 + 0.797 with the sum conserved. The final lines show why: permuting destroys a correlation of +1.000, scoring the model on rows that cannot exist.

Listing 8 — Fairness auditing: unawareness fails, and the audit depends on the ground truth

```
"""Listing 8: fairness auditing -- unawareness fails, and the audit depends on the
ground truth."""
import numpy as np
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(5)
n = 20000
group = rng.integers(0, 2, n)                    # protected attribute A (0 / 1)
skill = rng.normal(0, 1, n)                      # true qualification, same
distribution in both groups
zipc  = 0.9 * group + rng.normal(0, 0.5, n)      # proxy feature: strongly encodes
group
merit = (skill + rng.normal(0, 0.3, n) > 0.4).astype(int)          # who is ACTUALLY
qualified
y     = np.where(rng.random(n) < 0.45 * (1 - group), 0, merit)
# history erased 45% of g0 hires

X = np.c_[skill, zipc]                           # protected attribute NOT used as a
feature
(Xtr, Xte, ytr, yte, gtr, gte,
 mtr, mte) = train_test_split(X, y, group, merit, random_state=0)
clf = GradientBoostingClassifier(random_state=0).fit(Xtr, ytr)   # trained on BIASED
labels
pred = clf.predict(Xte)
```

```
def audit(yhat, ytruth, g, label):
    r0, r1 = yhat[g == 0].mean(), yhat[g == 1].mean()            # selection rates
    tpr = [yhat[(g == k) & (ytruth == 1)].mean() for k in (0, 1)]
    fpr = [yhat[(g == k) & (ytruth == 0)].mean() for k in (0, 1)]
    print(f"{label}")
    print(f"  selection rate   g0 {r0:.3f}  g1 {r1:.3f}   dem-parity diff {r1-r0:+.3f}"
          f"   disparate impact {r0/r1:.2f}")
    print(f"  TPR              g0 {tpr[0]:.3f}  g1 {tpr[1]:.3f}   gap {tpr[1]-tpr[0]:+.
3f}")
    print(f"  FPR              g0 {fpr[0]:.3f}  g1 {fpr[1]:.3f}   gap {fpr[1]-fpr[0]:+.
3f}")

audit(pred, yte, gte, "audited against HISTORICAL labels (the biased ones):")
print()
audit(pred, mte, gte, "audited against TRUE merit:")

# mitigation: group-specific thresholds to equalize TPR on merit (equal opportunity)
proba = clf.predict_proba(Xte)[:, 1]
thr = {k: np.quantile(proba[(gte == k) & (mte == 1)], 0.20) for k in (0, 1)}
pred_eo = (proba > np.where(gte == 0, thr[0], thr[1])).astype(int)
print(f"\ngroup thresholds g0 {thr[0]:.3f} / g1 {thr[1]:.3f}  (aim: TPR ~= 0.80 on
merit)")
audit(pred_eo, mte, gte, "after equal-opportunity thresholding, against TRUE merit:")
```

Output:

```
audited against HISTORICAL labels (the biased ones):
  selection rate   g0 0.273  g1 0.331   dem-parity diff +0.058   disparate impact 0.82
  TPR              g0 0.754  g1 0.835   gap +0.081
  FPR              g0 0.161  g1 0.053   gap -0.108

audited against TRUE merit:
  selection rate   g0 0.273  g1 0.331   dem-parity diff +0.058   disparate impact 0.82
  TPR              g0 0.743  g1 0.835   gap +0.093
  FPR              g0 0.023  g1 0.053   gap +0.030

group thresholds g0 0.422 / g1 0.559  (aim: TPR ~= 0.80 on merit)
after equal-opportunity thresholding, against TRUE merit:
  selection rate   g0 0.303  g1 0.309   dem-parity diff +0.006   disparate impact 0.98
  TPR              g0 0.800  g1 0.793   gap -0.006
  FPR              g0 0.038  g1 0.041   gap +0.003
```

Without ever seeing the protected attribute, the model reconstructs historical bias through the zip-code proxy: disparate impact 0.82 and a 9-point TPR gap against truly qualified group-0 candidates. Note the audit against historical labels *inverts* the FPR story (−0.108) — contaminated ground truth contaminates the audit. Group-specific thresholds equalize opportunity (gap −0.006) and, here, repair demographic parity (0.98) as a side effect.

Pitfalls, comparisons and practical tips

Method selection at a glance:

Question	Method	Scope	Cost	Key failure mode
Which features matter overall?	Permutation importance	Global	n_features × repeats evals	Correlated features deflate each other

Question	Method	Scope	Cost	Key failure mode
Which features matter (free)?	Impurity (MDI)	Global	Free with trees	Cardinality bias; training-set artifact
What does this feature contribute given the others?	Drop-column	Global	Retrain per feature	Expensive; correlated twin hides value
What shape is the effect?	PDP + ICE	Global	grid × n evals	PDP averages away interactions; impossible points
Why this prediction? (principled)	SHAP (TreeSHAP if trees)	Local → global	Poly for trees; heavy agnostic	Background choice; correlated credit-splitting
Why this prediction? (fast, any modality)	LIME	Local	perturbations × evals	Instability; kernel-width arbitrariness
Does behavior differ by group?	Fairness metrics + group SHAP	Global	Cheap given predictions	Ground-truth labels themselves biased

The recurring pitfalls:

- **Trusting MDI rankings.** The default `feature_importances_` inflates high-cardinality features and reflects training-set structure. Habit: recompute with permutation importance on held-out data before presenting any ranking (Listing 1's noise feature outranking real features is the canonical demo).
- **Reading raw coefficients as importance.** Only after standardization, and never individually under collinearity — report the correlation structure or grouped effects instead (Listing 2).
- **Presenting a PDP without ICE.** A flat PDP means "no effect" *or* "two opposite effects canceling"; only ICE curves distinguish them (Listing 3).
- **Explaining probabilities when SHAP explained log-odds.** TreeSHAP for classifiers decomposes the margin; contributions are additive there and not in probability space. Say which space you're in.
- **Forgetting the background.** SHAP values answer "compared to what?" — a different reference sample changes every number. Choose it to match the question (population average vs a specific cohort) and document it.
- **One-off LIME.** A single LIME run with default kernel width is an anecdote. Rerun across seeds, report stability, and prefer boundary cases where the local surface is informative (Listing 6).
- **Any importance under correlation.** Permutation deflates, SHAP splits, coefficients see-saw. Cluster correlated features and interpret groups (Listing 7).
- **Fairness through unawareness.** Dropping the protected attribute while proxies remain changes nothing except your ability to audit — you often need the attribute *at audit time* to measure the gaps (Listing 8).

- **Auditing against biased labels.** If the historical labels encode the discrimination, equalized odds against those labels certifies the bias. Interrogate the ground truth first.
- **Confusing the model with the world.** Every method here explains *what the model does*, not *what causes what*. SHAP on a proxy feature is evidence about the model's reliance, never causal evidence about reality. Causal claims need causal designs (Chapter 33 returns to this).

Practical workflow for a tabular model: permutation importance for the global ranking (with error bars, on test data) → TreeSHAP beeswarm for direction and shape → ICE for the top features to catch interactions → SHAP force/waterfall plots for individual decisions that need justification → group-wise fairness metrics if decisions touch people. Total cost for a tree ensemble: minutes.

Interview questions and answers

Q1. Distinguish global from local interpretability with an example of each.

Global explains overall model behavior: "income and tenure are the strongest drivers of predicted churn across the portfolio" (permutation importance, mean |SHAP|, PDP). Local explains one prediction: "this customer's 0.92 churn score is driven by three support calls last month and a month-to-month contract" (SHAP values for that row, LIME). They can disagree: a feature can be globally minor but decisive for a subpopulation, and Listing 3's interaction shows a per-row effect that averages to zero globally.

Q2. Why is impurity-based feature importance biased, and toward what?

MDI credits each feature with the impurity reduction of its splits, and features offering more candidate split points — continuous features, high-cardinality categoricals — get more chances to find a spuriously good split, so they accumulate importance even when uninformative. It's also computed on training data, so it reflects overfitting. In Listing 1 a continuous noise column ties two real features by MDI (0.074) while permutation importance on held-out data correctly zeroes it. The interviewer listens for: cardinality bias + training-data artifact + the fix (permutation on test data).

Q3. Explain permutation importance. What are its two main weaknesses?

Shuffle one feature's column on held-out data — destroying its relationship to the target while keeping its marginal distribution — and measure the metric drop; repeat and average. Weakness one: cost, one full evaluation pass per feature per repeat. Weakness two: correlated features — the model recovers the shuffled feature's signal from its correlated twin so the drop understates importance, and the shuffled rows are off-manifold combinations the model never trained on (Listing 7: importance 1.638 → 0.478 after adding a duplicate; correlation +1.000 → −0.024 after permuting).

Q4. When are linear-model coefficients a valid importance measure?

When features are standardized to a common scale (coefficients have units — per-dollar vs per-decade differences swamp real differences, Listing 2's 1,500× illusion) and features are not strongly collinear (under collinearity only the combined effect is identified; individual coefficients swing and flip sign across refits while their sum stays stable). Also, coefficients measure association given the other features in the model — dropping or adding covariates changes them.

Q5. What is a partial dependence plot and what independence assumption does it make?

PDP of feature j at value v = the average prediction when x_j is set to v for every row, swept over a grid — it shows the shape of the marginal effect. It assumes feature j is independent of the others: the averaging fabricates rows combining x_j = v with covariate values that never co-occur (forcing weight = 45 kg on tall people), so under correlation the curve is built on impossible points and can be arbitrary there.

Q6. Your PDP for a feature is flat. What are the two possible explanations and how do you tell them apart?

Either the feature truly has no effect, or it has strong effects of opposite sign in different subpopulations that cancel in the average. Plot ICE curves: flat ICE everywhere means no effect; ICE curves fanning into crossing bundles means an interaction the PDP erased. Listing 3 stages the second case — PDP slope −0.01 while the two groups have slopes −1.68 and +1.69.

Q7. Define the Shapley value and name the four axioms.

The Shapley value of feature j is its marginal contribution to the coalition payout $v(S\cup\{j\}) - v(S)$, averaged over all coalitions S with the weight $|S|!(|F|-|S|-1)!/|F|!$, which equals averaging over all orderings in which features could be revealed. Axioms: efficiency (values sum to $f(x) - E[f(X)]$), symmetry (interchangeable features get equal credit), dummy (a feature that changes no coalition's value gets zero), additivity (values for a sum of models = sum of values). It is the unique attribution satisfying all four.

Q8. What does SHAP's efficiency (local accuracy) property give you that LIME does not?

The SHAP values of one prediction sum exactly to the prediction minus the base value — a complete accounting: you can present "base rate 20%, +30 points from utilization, −8 from tenure, = 42%" and it adds up. LIME's coefficients are slopes of a local surrogate; they satisfy no summation constraint, so they rank and describe direction but never decompose the prediction. Listing 4's check: base + $\Sigma\varphi$ = 5.213 = f(x) exactly.

Q9. Why is computing exact Shapley values hard in general, and how do KernelSHAP and TreeSHAP each get around it?

The definition sums over all 2^d coalitions, each requiring an expectation over the background — exponential in features. KernelSHAP recasts Shapley values as the solution of a weighted linear regression over sampled coalitions (the Shapley kernel makes the regression solution equal the Shapley value), trading exactness for sampling error; it stays model-agnostic. TreeSHAP exploits tree structure to compute the exact values in polynomial time, $O(T \cdot L \cdot D^2)$ per instance, by tracking how coalitions flow through splits — exact and fast, but trees only.

Q10. For a tree classifier, in what space are TreeSHAP values additive, and why does it matter?

Log-odds (margin) space. The sigmoid is nonlinear, so contributions cannot be additive in probability — a +1 log-odds contribution moves p a lot near 0.5 and almost none near 0.99. Report and plot in log-odds when doing arithmetic; convert only the final sum (Listing 5: base $-0.012 + \Sigma\ -4.911 = -4.924 \rightarrow \sigma(-4.924) = 0.007$). Presenting SHAP bars in "probability points" that don't sum to the prediction is a common, detectable error.

Q11. How does the choice of background dataset change SHAP values?

SHAP answers "why does f(x) differ from E[f(X)] under this background" — the reference defines both the base value and every conditional expectation v(S). Explaining a loan denial against the population average gives different attributions than against previously approved applicants. There is no universally right background: match it to the contrastive question being asked and hold it fixed across explanations you intend to compare. Small backgrounds also add Monte-Carlo noise in KernelSHAP.

Q12. Walk through the LIME algorithm step by step.

(1) Generate perturbations around the instance: Gaussian noise in standardized feature space for tabular; randomly masking words/superpixels for text/images. (2) Query the black box for each perturbed point. (3) Weight points by a proximity kernel π_x (e.g. $\exp(-d^2/w^2)$) so nearby behavior dominates. (4) Fit a sparse/regularized weighted linear model to the (perturbation, prediction) pairs; its coefficients are the explanation, and the weighted R^2 tells you whether the local linear story is even adequate. Listing 6 is the full algorithm in ~15 lines.

Q13. Why are LIME explanations unstable, and what do you do about it?

Three sources: random perturbations (finite-sample noise), the arbitrary kernel width (too narrow = noisy fit, too wide = no longer local), and off-manifold sampling (perturbations may be unrealistic points where the model is undefined-ish). Mitigations: many perturbations, rerun across seeds and report whether top features persist (Listing 6 does 5 seeds), tune kernel width to a stable regime, check the surrogate's weighted R^2 before trusting the coefficients, and prefer explaining points where the model isn't saturated.

Q14. SHAP vs LIME — give a decision rule.

Tree ensemble on tabular data: TreeSHAP, no contest — exact, fast, axiomatic. Expensive black box where KernelSHAP is too slow, or text/image inputs where perturbation semantics are natural: LIME (or its descendants). Need a complete decomposition that sums to the prediction, or global aggregation from locals: SHAP. Need only a quick ranked sanity check of one prediction: LIME is fine, run it several times. Both inherit the correlated-features caveat; neither is causal.

Q15. How do you get a global explanation out of a local method like SHAP?

Compute SHAP values for a representative sample and aggregate: mean |SHAP| per feature = global importance; a beeswarm plot (dots = rows, x = SHAP value, color = feature value) adds direction and nonlinearity; SHAP dependence plots (feature value vs SHAP value) recover PDP-like shapes with interaction coloring. Listing 5 shows mean-|SHAP| ranking matching permutation importance exactly — and when the two disagree, suspect correlated features or a feature that matters intensely for few rows.

Q16. Two features in your model are near-duplicates. Describe what happens to permutation importance, SHAP, and coefficients.

Permutation: both deflate — shuffle one and the twin covers, so measured drops understate reality (1.638 → 0.478 in Listing 7). SHAP: credit splits roughly evenly between the twins with the total conserved by efficiency (1.659 → 0.864 + 0.797). Coefficients: unidentified individually — they trade off against each other across refits, signs can flip, only the sum is stable (Listing 2). Fix: group correlated features (cluster by correlation), interpret at group level, or drop one twin before interpreting.

Q17. What is the difference between explaining the model and explaining the world?

All methods here are descriptions of the fitted function f — which inputs move its output. That is not causality: the model may lean on a proxy (zip code) because it correlates with the true cause (historical discrimination, or income), and SHAP will faithfully report the model's reliance on the proxy. Acting on the world ("change the feature to change the outcome") requires causal assumptions the model never encodes. Saying "SHAP shows X causes Y" in an interview is an instant red flag; "SHAP shows the model relies on X" is correct.

Q18. Define demographic parity, equalized odds, and equal opportunity.

Demographic parity: equal selection rates across groups, P(Ŷ=1|A=a) constant in a — ignores the labels entirely. Equalized odds: equal TPR and equal FPR across groups — errors balanced conditional on the true outcome. Equal opportunity: the TPR half only — among truly qualified/positive individuals, equal chance of being selected. Parity suits contexts where the label itself is suspect or aspiration is equal representation; odds/opportunity suit contexts where the label is trusted and you want equal error burden.

Q19. What is the 80% (four-fifths) rule?

A disparate-impact screening threshold from US employment guidelines: the selection rate of the disadvantaged group divided by that of the advantaged group should be at least 0.8; below that, the practice is presumptively discriminatory and needs justification. It's a legal heuristic, not a statistical test — Listing 8's model sits at exactly 0.82, "passing" while still carrying a 9-point TPR gap against qualified group-0 candidates, which is why one metric is never the audit.

Q20. Why doesn't removing the protected attribute from the features make a model fair?

Because correlated proxies (zip code, name, school, purchase history) reconstruct it — Listing 8's model never sees the group yet reaches disparate impact 0.82 through the zip proxy, having learned from labels where 45% of one group's positives were erased. Worse, unawareness removes your ability to *measure* and *correct* gaps: group-aware auditing and mitigation typically need the attribute at evaluation time. The bias lives in the data-generating process, not in one column.

Q21. Calibration within groups and equalized odds — can you have both?

Generally no: the impossibility results (Kleinberg et al. 2016; Chouldechova 2017) show that when base rates differ across groups, a classifier cannot simultaneously be calibrated within each group and have equal TPR and FPR across groups, outside trivial cases (perfect prediction or equal base rates). Practical consequence: fairness is a set of mutually incompatible desiderata; you must argue from the application which criterion binds — and be ready to defend the choice, since optimizing one provably sacrifices another.

Q22. Your fairness audit uses historical outcome labels. What is the risk?

If historical decisions were biased, the labels encode the bias, and metrics conditioned on them are distorted: Listing 8's model shows an FPR gap of −0.108 against historical labels — apparently over-selecting the disadvantaged group — while against true merit the same predictions show a +0.093 TPR gap against it. Equalizing odds against poisoned labels certifies discrimination. Defenses: interrogate label provenance, use outcome measures less mediated by past decisions, model the label bias explicitly, or evaluate on cohorts where decisions were randomized/consistent.

Q23. Name the three mitigation families for unfair models with one example each.

Pre-processing: fix the data — reweighing samples, repairing feature distributions, removing proxies (hard: proxies hide in combinations). In-processing: fix the training — add a fairness penalty to the loss, adversarial debiasing where an adversary tries to predict the group from representations. Post-processing: fix the outputs — group-specific decision thresholds; Listing 8 equalizes TPR with per-group thresholds, collapsing the opportunity gap from +0.093 to −0.006 without touching the model. Post-processing is cheapest and auditable, but requires the attribute at decision time, which some legal regimes restrict.

Q24. What is a surrogate model in the global sense, and what must you always report with it?

Fit an interpretable model (shallow tree, sparse linear) to the black box's *predictions* — not the labels — and read the surrogate as a description of the black box. Always report fidelity: the surrogate's R^2/accuracy at mimicking the black box on held-out data. A surrogate with 0.6 fidelity describes 60% of the model's behavior; conclusions from it about the remaining 40% are fiction. Same rule locally: Listing 6 reports the weighted R^2 (0.623) alongside the LIME coefficients.

Q25. A regulator asks why customer 4711's loan was denied. Sketch your answer pipeline.

(1) TreeSHAP (or KernelSHAP) values for that application against a documented reference population; (2) present the top signed contributions in log-odds converted at the end to probability, verifying additivity; (3) translate to reason codes — the standardized adverse-action categories — by mapping the top negative contributors; (4) counterfactual check: verify the cited features actually flip the decision when improved (a reason code that can't change the outcome is not actionable); (5) stability check across the model's retraining history. Interviewer wants: reference-dependence, additivity, actionability, not just "run SHAP".

Q26. Write pseudocode for permutation importance.

Given model f, held-out (X, y), metric m: `base = m(y, f(X)); for j in features: scores = []; for r in 1..R: Xp = X.copy(); Xp[:, j] = shuffle(Xp[:, j]); scores.append(base - m(y, f(Xp))); importance[j] = mean(scores), std(scores)`. Key details: held-out data (not train), multiple repeats with reported spread, metric must match what you care about (importance under AUC ≠ importance under log loss), and correlated features need grouped shuffles.

Q27. Why can two models with identical accuracy have very different SHAP explanations, and what does that imply for trust?

Underdetermination (the "Rashomon effect"): many functions fit the same data equally well while using different features — especially under correlated features, where models can trade one twin for the other freely. Retrain with a different seed and the explanation can shift even as the metric holds. Implication: an explanation describes *this* model instance, not the task; for consequential decisions check explanation stability across retrains/seeds, and treat features whose importance persists across the Rashomon set as more trustworthy than any single ranking.

Q28. You need to justify predictions in a high-stakes setting. Argue interpretable-by-design vs black-box + post-hoc.

For high stakes, the burden of proof favors intrinsic interpretability: a sparse scoring system or shallow tree is its own explanation — exact, stable, contestable — while post-hoc explanations are approximations that can be unfaithful, unstable (LIME), or reference-dependent (SHAP), and can rationalize a flawed model persuasively. The counterargument: on complex signals the accuracy gap costs real outcomes, and TreeSHAP on a GBM is exact-for-the-model and auditable. Defensible synthesis: first *measure* the accuracy gap — on many tabular problems it's near zero (fit the interpretable model, compare honestly); pay for a black box only when the gap is material, and then budget for explanation validation, stability testing, and fairness audits as first-class deliverables.